

Rapport projet d'apprentissage par renforcement

TOMIL SHI

Année universitaire 2025-2026

Introduction :

Pour mon projet d'apprentissage par renforcement, j'ai décidé de travailler sur le jeu Atari « Assault ». C'est un jeu de tir classique où l'objectif est de contrôler un vaisseau spatial fixe en bas de l'écran pour détruire des petits vaisseaux déployés par un vaisseau-mère tout en haut de l'écran qui essaye de détruire notre vaisseau. On doit donc esquiver les attaques du vaisseau-mère tout en détruisant les petits vaisseaux déployés.

C'est donc intéressant pour une IA de travailler dessus car elle doit apprendre à anticiper les trajectoires des attaques tout en détruisant les petits vaisseaux.

Pour résoudre ce problème, j'ai décidé d'implémenter un Deep Q-Network (DQN). La raison principale de ce choix est la nature des données d'entrée. En effet, dans le jeu Assault, l'état du jeu n'est pas donné sous forme de coordonnées précises mais sous forme d'une image de pixel, c'est-à-dire l'écran du jeu. Avec un Q-learning classique, le nombre d'états possibles est beaucoup trop grand, il est donc impossible de stocker une valeur pour chaque combinaison de pixels. Le DQN résout ce problème en remplaçant la table Q par un réseau de neurones convolutif.

De plus, l'espace d'action est discret, le nombre d'actions est petit, il y a 7 actions en tout, ce qui est parfait pour le DQN puisqu'il prédit une valeur Q pour chaque action possible et choisit la meilleure.

Enfin, dans une partie d'Assault, deux images consécutives sont quasi identiques. Si on entraîne le réseau « en direct », ces corrélations finissent par biaiser l'agent qui oublie ce qu'il a appris précédemment. L'utilisation du mécanisme d'Experience Replay propre au DQN m'a permis de casser la corrélation. En effet, en stockant les transitions dans une mémoire tampon et en entraînant le réseau sur des échantillons aléatoires, j'ai pu casser la corrélation temporelle.

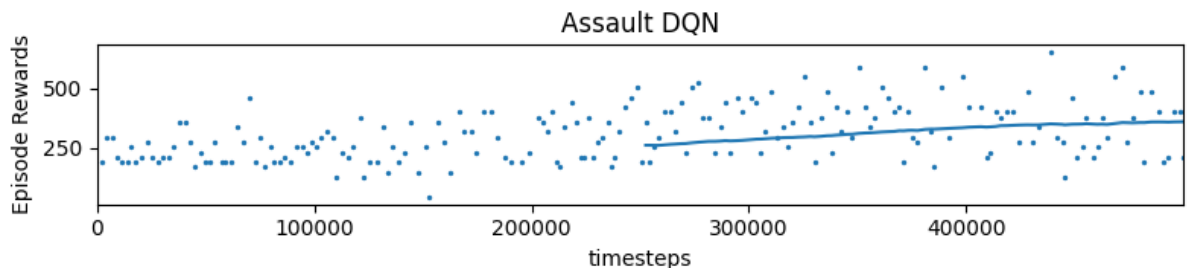
Recherche des meilleurs hyperparamètres :

Une fois l'algorithme DQN choisi, j'ai joué avec les hyperparamètres pour avoir les meilleurs résultats possibles. J'ai donc fait des essais avec 500 000 timesteps vu que c'est assez long d'entraîner le modèle.

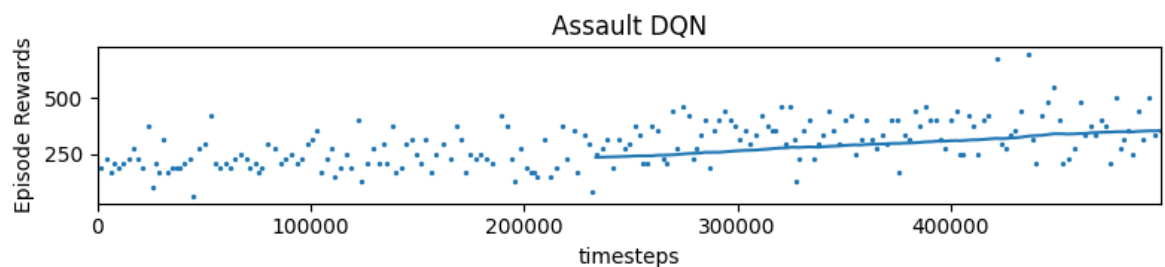
Choix du learning rate :

J'ai testé trois valeurs de learning rate :

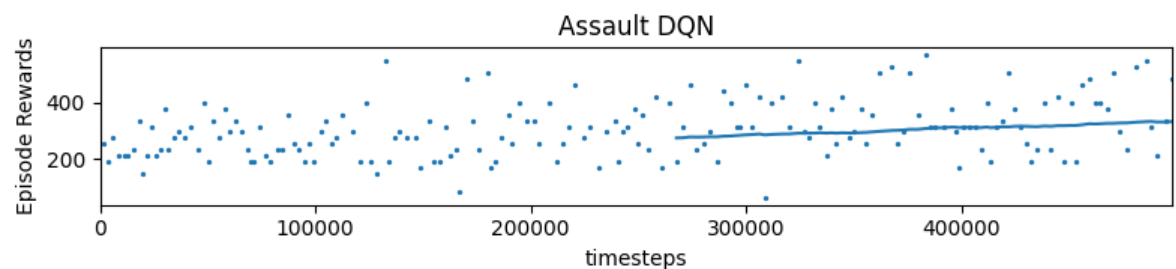
- Learning rate = 0.001 :



- Learning rate = 0.0001 :



- Learning rate = 0.00001 :



En regardant les récompenses de chaque modèle pendant l'entraînement, on peut observer que les modèles avec un learning rate de 0.001 et 0.0001 sont meilleurs que celui avec un learning rate de 0.00001. Etant donné qu'avec un learning rate de 0.001 ou 0.0001, les résultats sont assez

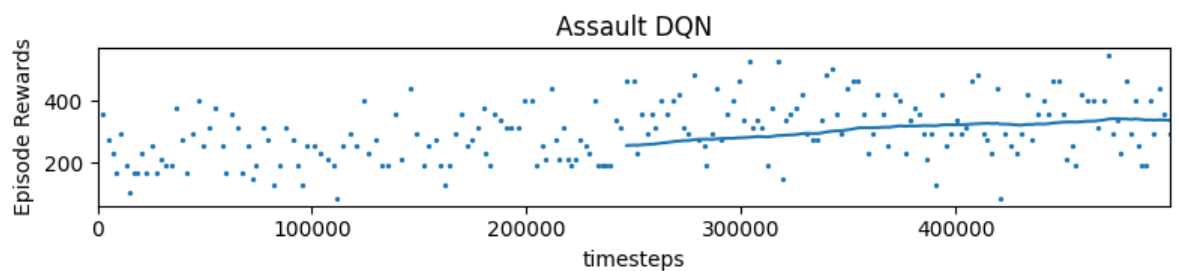
similaires, on va décider de choisir un learning rate de 0.0001 pour notre modèle final.

Choix du buffer size :

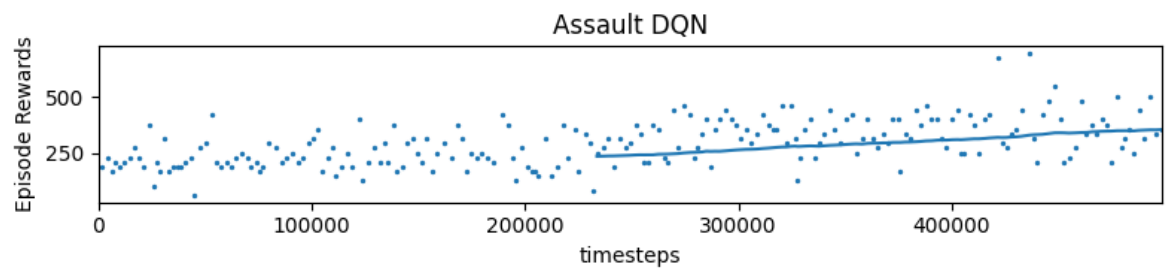
J'ai ensuite testé plusieurs valeurs de buffer size, de 30000 à 50000, il n'y avait pas vraiment de différence donc au final j'ai choisi un buffer size de 50000.

Choix de exploration fraction :

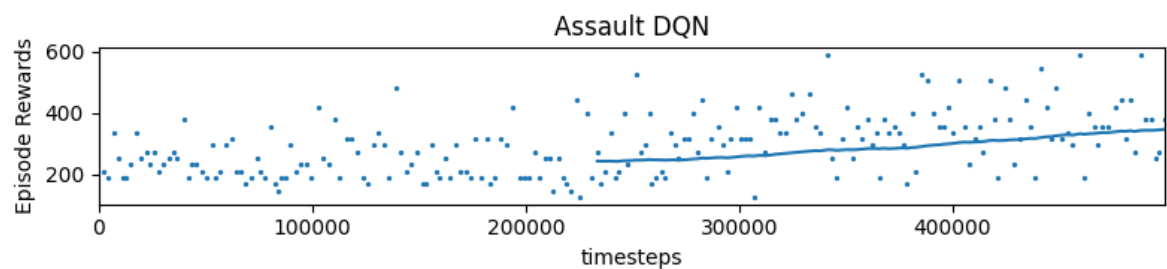
- Exploration fraction = 0.05 :



- Exploration fraction = 0.1 :



- Exploration fraction = 0.2 :



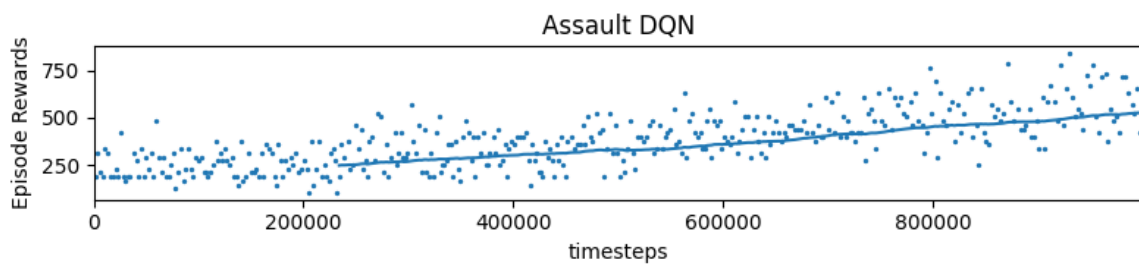
On observe que le modèle avec une valeur de 0.1 est le meilleur. On va donc choisir exploration fraction = 0.1

Modèle final :

Le modèle final aura donc comme paramètre :

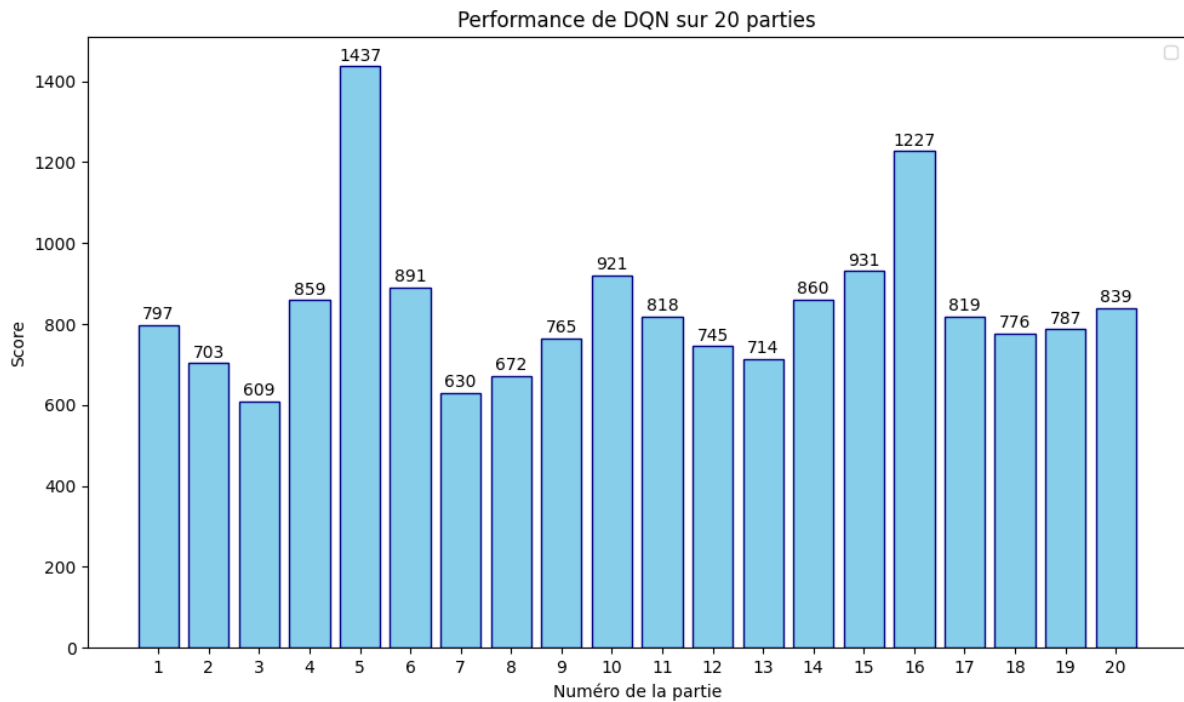
- Learning rate = 0.001
- Buffer size = 50000
- Exploration fraction = 0.1

J'ai décidé d'entraîner le modèle final sur 1 million de timesteps et j'obtiens le graphique suivant pour l'entraînement :



Avec ce graphique, on observe que l'agent apprend bien au fil des timesteps avec une moyenne à la fin d'environ 500 points et des pics à environ 800 points.

En faisant maintenant jouer le modèle sur 20 parties, on obtient le graphique suivant :



On observe que l'agent a en moyenne un score d'environ 750-800 points et a des pics à 1437 et 1227 points, ce qui n'est pas mal du tout. L'agent arrive à survivre et à abattre ses ennemis de manière correcte.

Conclusion :

Ce projet sur le jeu Assault de l'environnement Atari a été une excellente opportunité de mettre en pratique les concepts de l'apprentissage par renforcement. En utilisant un DQN, j'ai obtenu un agent capable de jouer au jeu Assault de manière assez correcte en seulement 1 million timesteps. Le modèle a donc encore une marge de progression importante en l'entraînant beaucoup plus. Cependant, avec le manque de GPU et le temps nécessaire assez long pour entraîner le modèle, ce n'était pas possible de l'entraîner avec plus d'un million timesteps.